AgentReel

How to set it up, how to run it, and what happens inside when you press Generate.

AgentReel turns a short brief (and optionally a PDF or DOCX) into a finished 30–60 second vertical video. Seven AI agents plan, research, write, storyboard, build the assets, edit and check the video. Remotion renders it, Manim draws the diagrams, and FFmpeg finishes the audio and file. There are three styles: **Motion graphics** (animated explainer with diagrams), **Minecraft parkour** and **Subway runner** (narration and big captions over gameplay).

1. What you need

Tool	Version used	Why
Windows 10/11	Windows 11	Tested platform (SAPI voice fallback is Windows-only)
Python	3.12 (3.11+)	The pipeline, agents and web server
Node.js + npm	22 / 11	Remotion, which renders every video
FFmpeg + FFprobe	9.0, on PATH	Cutting footage, loudness, final encoding, checks
Internet	–	AI provider and the Edge text-to-speech voices
An AI key	Gemini (free) or OpenRouter	Optional: without one, simple local templates are used

Manim is installed by `pip`; no LaTeX is needed. Remotion downloads its own headless Chrome on the first render.

2. One-time setup

```
git clone <your repo url> AgentReel
cd AgentReel
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
cd remotion; npm install; cd ..
copy .env.example .env
```

Open `.env` and paste your key: `GEMINI_API_KEY=...` (from Google AI Studio) and/or `OPENROUTER_API_KEY=...`. `LLM_PROVIDER=gemini` makes Gemini the default.

For the gameplay styles, fetch and check the clips once (they are not in the repository):

```
.\.venv\Scripts\python.exe scripts\setup_gameplay_assets.py
.\.venv\Scripts\python.exe run.py --status      # shows tools, keys and footage readiness
```

3. Running it

The web app (main way)

```
.\.venv\Scripts\python.exe web.py --provider gemini
```

Your browser opens `http://127.0.0.1:8600`. Use `--provider openrouter` to switch provider, `--port 8700` for another port and `--no-browser` to skip opening a tab. Stop it with Ctrl+C. Only one copy should run on a port at a time.

The command line

```
# any topic
.\.venv\Scripts\python.exe run.py --topic "Why do cats purr?" --mode og --duration 45 --
provider gemini

# a YAML brief with a source document (see briefs\examples)
.\.venv\Scripts\python.exe run.py --brief briefs\examples\green_hydrogen.yaml --seed 42

# continue a stopped video, or restyle captions of a finished one
.\.venv\Scripts\python.exe run.py --resume <job id>
.\.venv\Scripts\python.exe run.py --rerender <job id> --highlight-color "#FF3D7F"
```

Flag	What it does
`--mode og|minecraft|subway`	The style
`--duration 30-60`	Target length in seconds
`--voice en-IN-PrabhatNeural`	Narration voice (`--list-voices` lists them)
`--provider gemini|openrouter`	Which AI to use for this run
`--seed N`	Makes local choices repeatable (colours, footage cuts, looks)
`--fresh-script`	Write a new script even if this brief was used before
`--local-only` / `--ai-required`	Never call the AI / fail if the AI is unavailable
`--dev-short`	A quick small test render, clearly labelled as a test

Using the web app

1. **Create:** type what the video should explain, optionally drag in a PDF/DOCX, pick a style, length, format, type and voice. "More options" holds audience, tone, must-mention, avoid, ending line and the AI setting. Press Generate (or Ctrl+Enter).
2. **Rendering:** a live preview of your video's plan, the percentage (Remotion reports real frame counts), and each step. Writing steps are tagged *AI*, *reused* or *template*.
3. **Result:** play, download (a real Save-as window), captions file, rename, delete, and "Try another style" to make a new version that keeps the same script.

4. **Library:** all your videos, grouped by version, with search, style tabs and delete with Undo. Deleted videos go to `outputs\_trash`.

4. What happens when you press Generate

Planner → Researcher → Scriptwriter → Storyboarder → Voice (TTS) → Visuals → Manim → Editor → Remotion → FFmpeg → Critic / QA

The full agent graph is on the next page.

#	Agent / stage	What it does	Writes
1	Planner	Decides the structure: number of scenes, which beats deserve a diagram, what research is needed.	`plan.json`
2	Researcher (RAG)	Splits your documents into passages, finds the relevant ones with BM25 search, and turns them into facts. Every claim is checked against the source text; numbers must appear word for word, otherwise the fact is marked unsupported.	`facts.json`
3	Scriptwriter	Writes the narration in short lines sized to the length. Each factual line cites its fact ids; lines with numbers no fact contains are removed.	`script.json`, `script.md`
4	Storyboarder	Groups lines into scenes and picks a visual for each: title, key points, chart, timeline, a Manim animation, an end card, and an animated background. Invented text, links or contacts are stripped.	`storyboard.json`
5	Asset Generator	Records the voice line by line (Edge TTS, with word timings), measures every clip, renders Manim animations, and cuts gameplay footage from the middle of the clips.	`assets\`
6	Editor	Builds the exact timeline from the measured audio: scene timings, captions synced to words, SRT file, and the props Remotion renders.	`render_props.json`, `captions.srt`
–	Remotion + FFmpeg	Remotion renders the video; FFmpeg evens the loudness to about −14 LUFS and encodes H.264/AAC.	`final.mp4`
7	Critic / QA	Runs about 30 checks (format, length 30–60 s, sync, black frames, loudness, captions match narration, every claim supported…) and, with an AI, reviews the content. A failure is sent back to the stage that caused it, at most twice.	`qa_report.json`

How the AI is used, and what happens without it

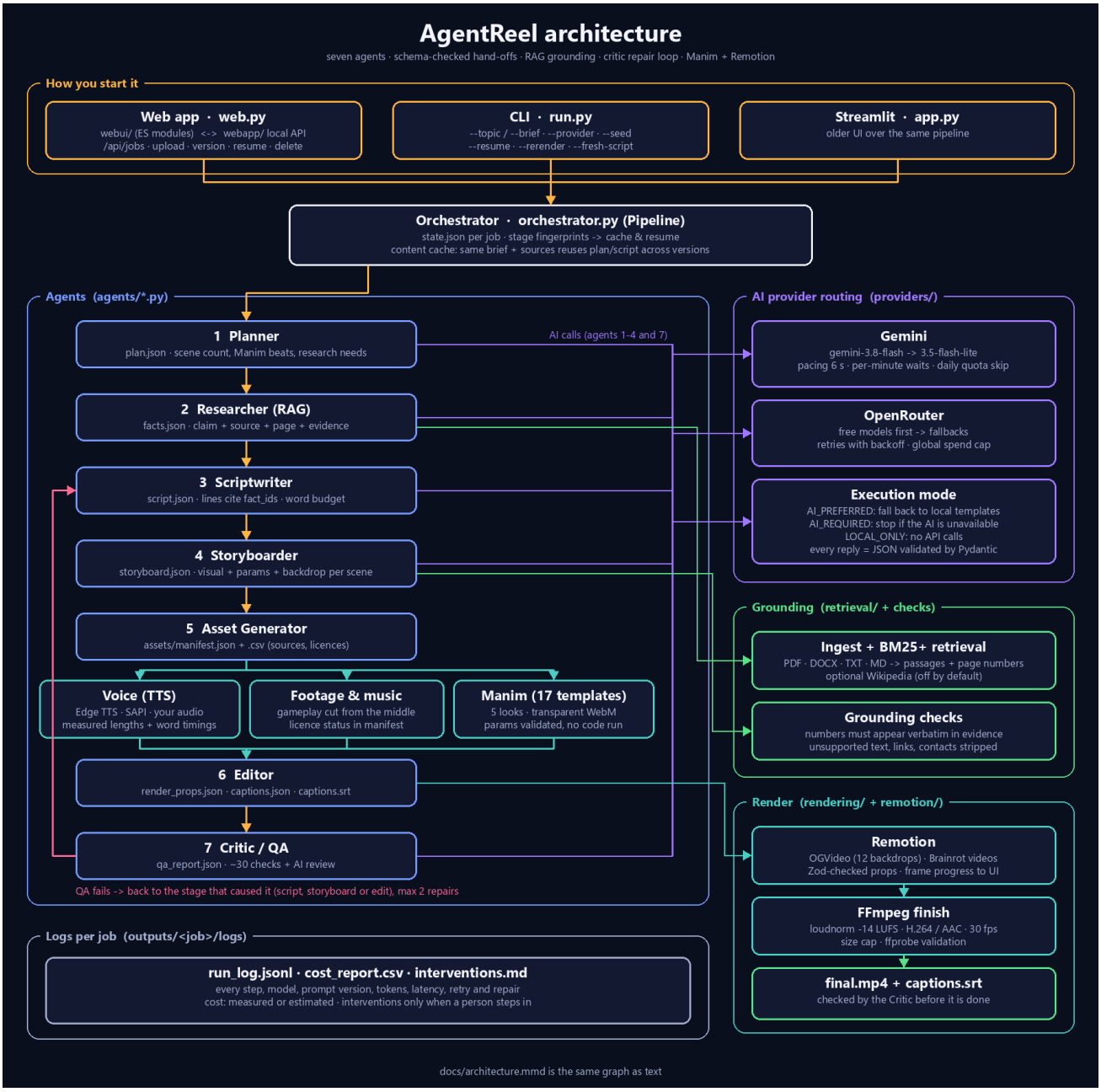
The AI only plans, writes, storyboards and reviews; timing, rendering and checking are ordinary code. Every AI reply must be valid JSON matching a schema, or it is retried. If the provider fails or its free daily limit is used up, **AI preferred** mode falls back to simple local templates (marked *template* on the Rendering screen), **AI only** stops with an error, and **Local only** never calls the AI. Gemini requests are

paced to stay inside the free tier; a model whose daily quota is spent is skipped until midnight Pacific time.

Caching, resume and versions

Each stage stores a fingerprint of its inputs. If you resume or re-render, finished stages are reused and only what changed runs again. Making a new version of a video (another style or voice) reuses the same plan and script, so it is quick and consistent.

Agent graph


AgentReel architecture
seven agents · schema-checked hand-offs · RAG grounding · critic repair loop · Manim + Remotion
How you start it
Web app · web.py
webui/ (ES modules) <-> webapp/ local API
/api/jobs · upload · version · resume · delete
CLI · run.py
--topic / --brief · --provider · --seed
--resume · --rerender · --fresh-script
Streamlit · app.py
older UI over the same pipeline
Orchestrator · orchestrator.py (Pipeline)
state.json per job · stage fingerprints -> cache & resume
content cache: same brief + sources reuses plan/script across versions
Agents (agents/*.py)
1 Planner
plan.json · scene count, Manim beats, research needs
2 Researcher (RAG)
facts.json · claim + source + page + evidence
3 Scriptwriter
script.json · lines cite fact_ids · word budget
4 Storyboarder
storyboard.json · visual + params + backdrop per scene
5 Asset Generator
assets/manifest.json + .csv (sources, licences)
Voice (TTS)
Edge TTS · SAPI · your audio
measured lengths + word timings
Footage & music
gameplay cut from the middle
licence status in manifest
Manim (17 templates)
5 looks · transparent WebM
params validated, no code run
6 Editor
render_props.json · captions.json · captions.srt
7 Critic / QA
qa_report.json · ~30 checks + AI review
QA fails -> back to the stage that caused it (script, storyboard or edit), max 2 repairs
AI calls (agents 1-4 and 7)
AI provider routing (providers/)
Gemini
gemini-3.8-flash -> 3.5-flash-lite
pacing 6 s · per-minute waits · daily quota skip
OpenRouter
free models first -> fallbacks
retries with backoff · global spend cap
Execution mode
AI_PREFERRED: fall back to local templates
AI_REQUIRED: stop if the AI is unavailable
LOCAL_ONLY: no API calls
every reply = JSON validated by Pydantic
Grounding (retrieval/ + checks)
Ingest + BM25+ retrieval
PDF · DOCX · TXT · MD -> passages + page numbers
optional Wikipedia (off by default)
Grounding checks
numbers must appear verbatim in evidence
unsupported text, links, contacts stripped
Render (rendering/ + remotion/)
Remotion
OGVideo (12 backdrops) · Brainrot videos
Zod-checked props · frame progress to UI
FFmpeg finish
loudnorm -14 LUFS · H.264 / AAC · 30 fps
size cap · ffprobe validation
final.mp4 + captions.srt
checked by the Critic before it is done
Logs per job (outputs/<job>/logs)
run_log.jsonl · cost_report.csv · interventions.md
every step, model, prompt version, tokens, latency, retry and repair
cost: measured or estimated · interventions only when a person steps in
docs/architecture.mmd is the same graph as text

5. Animations and backgrounds

Manim animations (picked by the Storyboarder when the topic fits): process steps, flow diagram, timeline, bar chart, line chart, donut chart, function graph, step-by-step equation, chemical reaction, sorting/searching algorithm (runs the real algorithm), AI agent loop, cybersecurity attack/defence, network graph, cycle, Venn diagram, pyramid and mind map. Each Manim scene also gets one of five looks (rounded, pill, sharp, glass, underline) so diagrams in one video do not look alike.

Backgrounds (one per scene, never the same twice in a row): orbs, grid, waves, rays, shapes, aurora, constellation, hexgrid, circuit, contours, stripes and matrix.

6. Where everything is saved

```
outputs\<job id>\
  final.mp4  captions.srt  qa_report.json  state.json
  brief.json  plan.json  facts.json  script.json  storyboard.json  render_props.json
  assets\    voice clips, Manim clips, gameplay background, manifest.csv (sources and
licences)
  logs\      run_log.jsonl (every step and AI call)  cost_report.csv  interventions.md
(manual steps only)
```

7. Limits and tips

- **Free AI quotas:** Gemini's main free model allows about 20 requests a day (a video uses about 6–8). When it runs out, a lighter backup model is used, which writes shorter scripts.
- **Length:** production videos must be 30–60 s; the QA report fails anything outside that.
- **Gameplay licences:** the bundled Minecraft and Subway Surfers clips are marked *needs review*. For anything public, use Motion graphics or footage you recorded yourself.
- **Live demos:** use `--fresh-script` (or "Write a fresh script" in More options) so the script is written on the spot rather than reused.
- **Facts:** numbers and contacts only appear if a source contains them; without sources a factual video stays general on purpose.

8. Troubleshooting

Symptom	Cause and fix
Rendering screen shows *template* steps	The AI was skipped (no key or daily limit). Check the reason under the steps; try again later or switch provider.
No AI calls in the terminal, wrong provider	Another copy of the app is on the same port. Close it and start `web.py --provider gemini` again.

Minecraft/Subway style is greyed out	No usable gameplay clip. Run `scripts\setup_gameplay_assets.py`, then `run.py --status`.
"Remotion is not installed"	Run `npm install` inside the `remotion` folder.
A video stopped part-way	Open it and press "Continue where it stopped", or `run.py --resume <job id>`.
Tests	`.\.venv\Scripts\python.exe -m pytest -m "not slow"` (fast) and `-m slow` (real renders); `cd remotion; npx vitest run`.